



DISTRITO GOURMET

Alex Vicente López

Ciclo de desarrollo de aplicaciones web

Memoria del Proyecto de DAW

IES Serra Perenxisa Curso 2025-2026

Periodo: marzo/junio

Tutor individual: Jose Luis Soriano Lopez

Resumen / Resum / Abstract

Español:

Distrito Gourmet es una plataforma web integral diseñada para optimizar la gestión de reservas y pedidos online en el sector de la restauración. Este proyecto resuelve la problemática de la dependencia de plataformas externas, eliminando el pago de comisiones y garantizando la soberanía total de los datos del negocio. Mediante una arquitectura moderna y desacoplada, utilizando Laravel para el backend y React junto con Vite para el frontend, se ha desarrollado una solución escalable. El resultado es una experiencia de usuario accesible, segura y de alto rendimiento.

Valencià:

Distrito Gourmet és una plataforma web integral dissenyada per a optimitzar la gestió de reserves i comandes online en el sector de la restauració. Aquest projecte resol la problemàtica de la dependència de plataformes externes, eliminant el pagament de comissions i garantint la sobirania total de les dades del negoci. Mitjançant una arquitectura moderna i desacoblada, utilitzant Laravel per al backend i React juntament amb Vite per al frontend, s'ha desenvolupat una solució escalable. El resultat és una experiència d'usuari accessible, segura i d'alt rendiment.

English:

Distrito Gourmet is a comprehensive web platform designed to optimize online reservation and order management in the restaurant sector. This project solves the problem of dependence on external platforms, eliminating commission fees and guaranteeing total sovereignty over business data. Through a modern and decoupled architecture, using Laravel for the backend and React along with Vite for the frontend, a highly scalable solution has been developed. The result is an accessible, secure, and high-performance user experience

Tabla de contenido

Resumen / Resum / Abstract	2
1. Introducción	5
2. Estado del arte	8
3. Estudio de viabilidad.	11
3.1. Análisis DAFO detallado del proyecto	11
3.2. Estudio de mercado y viabilidad	13
3.3. Viabilidad técnica y económica del proyecto	14
3.3.1. Viabilidad técnica	14
3.3.2. Viabilidad económica	14
3.3.3. Viabilidad temporal	15
4. Análisis de requisitos	17
4.1. Descripción de requisitos	17
4.1.1. Texto explicativo	17
4.1.2. Diagramas de caso de uso	18
5. Diseño	20
5.1. Diseño conceptual entidad-relación	20
5.2. Diseño lógico relacional.	20
5.3. Diseño físico o diagrama MySQL	21
5.4. Orientación a objetos	28
5.4.1. Diagrama de clases	28
5.4.2. Diagrama de secuencias.	29
5.4.3. Diagrama de actividad	29
5.4.4. Mapa web	30
5.4.5. Mockups	30
6. Codificación	31
6.1. Tecnologías elegidas y su justificación	31
6.2. Entorno servidor	31
6.2.1. Descripción general	31
6.3. Entorno cliente	32
6.3.1. Descripción general	32
6.3.2. Compatibilidad con navegadores	32
6.4. Documentación interna de código	32
6.4.1. Esquema de documentación (ejemplo backend)	32
6.4.2. Esquema de documentación (ejemplo frontend)	33
6.5. Documentación externa	33
6.5.1. Manual del usuario.	33
6.5.2. Documentación de la API	33
6.5.3. Manual de despliegue y mantenimiento.	33
6.6. Ejemplos de implementación	34
6.6.1. Rutas de la API	34
6.6.2. Lógica del backend	34
6.6.3. Modelo y relación	35

6.6.4. Consumo de la API desde el frontend	36
7. Despliegue	37
7.1. Diagramas de despliegue	37
7.2. Descripción de la instalación o despliegue	37
7.2.1. Fichero de configuración:	37
7.2.2. Descripción del servidor hosting utilizado.	38
8. Herramientas de apoyo	40
8.1. Control de versiones	40
8.2. Sistemas de integración continua	40
8.3. Gestión de pruebas	40
9. Conclusiones.	42
9.1. Conclusiones sobre el trabajo realizado	42
9.2. Conclusiones personales	42
9.3. Posibles ampliaciones y mejoras	43
10. Bibliografía	44
10.1. Artículos y apuntes	44
10.2. Direcciones web	44
11. Recursos del proyecto	46
Repositorio	46
Video de demostración	46

1. Introducción

El proyecto se origina como una respuesta directa y estratégica a una problemática estructural y persistente dentro del sector de la restauración: la ineficiente y costosa gestión de reservas de mesas. La práctica actual, predominante en la mayoría de los establecimientos, consiste en la adhesión y dependencia de plataformas externas de gestión de reservas, tales como TheFork, ElTenedor u OpenTable, lo cual, a pesar de ofrecer una solución digital inicial, introduce una serie de inconvenientes sustanciales y limitaciones operativas.

Análisis crítico de la dependencia de plataformas externas:

Aunque estos sistemas genéricos han facilitado la transición a la reserva digital, su implementación conlleva restricciones significativas que impactan negativamente la eficiencia y la autonomía del negocio:

1. **Carencia de personalización y adaptación operativa:** Los sistemas de terceros son, por naturaleza, soluciones "talla única". Esto implica que no se adaptan ni se configuran para reflejar las necesidades operativas específicas, la distribución física de mesas o las políticas particulares de turnos y sittings de cada restaurante.
2. **Dependencia tecnológica y de marca:** El restaurante desarrolla una marcada dependencia de servicios de terceros para una función operativa crítica. Esta dependencia se extiende a la tecnología, las actualizaciones y, crucialmente, a la percepción de la marca, ya que la experiencia de reserva del cliente está mediada por la interfaz de la plataforma externa.
3. **Impacto económico y restricción del control:** El modelo de negocio de estas plataformas se basa en el cobro de elevadas comisiones por reserva (cover charges), lo que erosiona significativamente el margen de beneficio del establecimiento. Adicionalmente, el restaurante pierde control total sobre sus propios datos de clientela (customer data), la gestión interna de la disponibilidad, e incluso puede derivar en errores operacionales como la sobre-reserva o duplicidades no sincronizadas con el sistema de gestión interno.
4. **Barreras en la comunicación y la fidelización:** La intermediación de la plataforma dificulta o impide el establecimiento de una comunicación directa, inmediata y eficiente con la clientela. Esta pérdida de contacto directo limita las

oportunidades para la personalización de la experiencia pre-visita y post-visita, obstaculizando estrategias efectivas de fidelización.

Objetivo fundamental y solución propuesta:

El objetivo fundamental y disruptivo de este proyecto es el desarrollo de una aplicación web propia, robusta y escalable para la gestión integral de reservas. Esta solución será diseñada a medida (tailor-made) para un restaurante específico, permitiendo un ajuste perfecto a su modelo operativo. Esta solución tecnológica aspira a alcanzar dos metas principales:

- **Optimización de la experiencia del cliente (front-end):** Ofrecer a los clientes un sistema ágil, intuitivo y sencillo para efectuar y modificar reservas desde cualquier dispositivo (diseño responsive), garantizando una experiencia de usuario superior a la de las opciones genéricas existentes.
- **Empoderamiento del administrador (back-end):** Suministrar a los administradores del restaurante herramientas centralizadas, potentes y de fácil manejo para la gestión en tiempo real de la disponibilidad de mesas, la configuración de horarios, la gestión de turnos y la consolidación de la información de clientes. Esto resultará en la eliminación completa de costes asociados a comisiones y la erradicación de la dependencia externa.

Alcance y desarrollo del proyecto:

A lo largo del proceso de desarrollo, se abordarán aspectos clave que aseguren la viabilidad y el éxito del proyecto:

- **Definición funcional:** Se identificarán y desarrollarán las funcionalidades esenciales requeridas tanto por los usuarios finales (clientes) como por los administradores (personal del restaurante), incluyendo sistemas de notificación, waitlists y mapas de mesas virtuales.
- **Seguridad y privacidad de datos:** Se implementarán estrategias de seguridad de vanguardia para proteger la información sensible de los clientes (cumplimiento estricto con normativas como el GDPR) y para garantizar la integridad y disponibilidad del sistema.

- **Metodología y Experiencia de Usuario (UX/UI):** Se aplicará una metodología centrada en el usuario para optimizar la interfaz (UI) y la experiencia de usuario (UX), asegurando que el proceso de reserva sea más rápido y claro que en las alternativas existentes.
- **Desafíos técnicos:** Se gestionarán los desafíos inherentes a la implementación, como la integración con otros sistemas de punto de venta (POS) del restaurante (si es necesario), la escalabilidad de la base de datos y la optimización del rendimiento del back-end.

Conclusión y valor estratégico:

En definitiva, este proyecto trasciende la mera satisfacción de una necesidad operativa. Representa una iniciativa estratégica para proporcionar al restaurante una herramienta de gestión flexible, eficiente y completamente adaptada, que no solo optimizará drásticamente la gestión interna, reduciendo costes y mejorando el control de datos, sino que también elevará la satisfacción y la experiencia digital de sus clientes, fortaleciendo la relación directa y la fidelidad hacia la marca.

2. Estado del arte

En el sector de la restauración, la gestión digital de reservas ha trascendido de ser una simple conveniencia a convertirse en un pilar estratégico y una necesidad fundamental para cualquier negocio que aspire a la excelencia en el servicio. La implementación de sistemas de reservas online no solo tiene como objetivo mejorar drásticamente la experiencia del cliente, sino también optimizar la operativa interna de los negocios, permitiendo una mejor planificación de la ocupación, la gestión del staff y la reducción de errores humanos.

Actualmente, el mercado está dominado por un ecosistema de diversas plataformas reconocidas que ofrecen servicios de reservas en línea, consolidándose como intermediarios esenciales. Entre ellas, destacan nombres como TheFork, OpenTable, Bookatable y Resy. Estas soluciones se han posicionado ofreciendo funcionalidades robustas que incluyen la gestión integral de reservas, la posibilidad de implementar ofertas y promociones dinámicas y el acceso a análisis detallados de clientes y hábitos de consumo.

No obstante, a pesar de su amplia adopción y sofisticación, estas plataformas presentan una serie de limitaciones significativas que merman su idoneidad, especialmente para los restaurantes locales e independientes:

1. **Falta de personalización y rigidez de adaptación:** La arquitectura y el diseño de la mayoría de estas plataformas están optimizados y estandarizados para servir a grandes cadenas de restauración o a un modelo de negocio genérico. Consecuentemente, demuestran ser inherentemente rígidas, no adaptándose con fluidez a las necesidades operativas específicas y únicas que caracterizan a cada restaurante local o de barrio.
2. **Costes operativos elevados y afectación a la rentabilidad:** El modelo de negocio predominante se basa en la imposición de comisiones por cada reserva procesada, o en la exigencia de suscripciones mensuales con cuotas elevadas. Este sistema de costes recurrentes y a menudo incrementales puede afectar seriamente la rentabilidad y el margen operativo, especialmente para los negocios pequeños o emergentes.
3. **Accesibilidad limitada e insuficiencia de localización:** Un número reducido de estas plataformas cumple rigurosamente con los estándares de accesibilidad

(WCAG), lo que puede excluir a usuarios con ciertas discapacidades. Además, la ausencia de interfaces multilingües adaptadas a contextos locales, como la inclusión del valenciano u otras lenguas cooficiales, representa una barrera idiomática y de inclusión cultural.

Tecnologías y lenguajes de desarrollo empleados en el sector

Las soluciones de gestión de reservas en el status quo se apoyan en un stack tecnológico web robusto y probado. Generalmente, su desarrollo se articula sobre tecnologías fundamentales como HTML, CSS y JavaScript para el frontend, y se apoya en bases de datos relacionales como MySQL para la persistencia de la información.

En el caso de las plataformas más avanzadas y escalables, se hace un uso extensivo de frameworks y bibliotecas modernas para asegurar una mejor experiencia de usuario (UX) y una escalabilidad del sistema:

- **Frameworks frontend** (p. ej., Vue.js, React o Angular) para construir interfaces de usuario dinámicas y reactivas.
- **Frameworks backend** (p. ej., Laravel, Django o Ruby on Rails) para gestionar la lógica de negocio, la seguridad y la interacción con la base de datos.
- **Bibliotecas de styling** (p. ej., Bootstrap o Materialize) para asegurar interfaces responsive y un diseño coherente.

Estrategias de implementación y enfoques de las plataformas líderes

Las plataformas líderes han implementado exitosamente estrategias enfocadas en la usabilidad y la integración: ofrecen interfaces responsive que se adaptan a cualquier dispositivo, potentes paneles de administración para los restauradores y la capacidad de conexión fluida con redes sociales y otros canales digitales. No obstante, su enfoque tiende a ser excesivamente genérico y poco flexible. Priorizan la estandarización y la eficiencia a gran escala, lo que choca con la aspiración de muchos negocios por lograr independencia tecnológica y una adaptación hiperlocal a su idiosincrasia y clientela. -----
Novedades y aportaciones diferenciales de Distrito Gourmet: Una propuesta local y autónoma.

La propuesta de Distrito Gourmet emerge como una alternativa disruptiva que busca solventar las carencias del mercado, ofreciendo una solución de reservas diseñada desde

la perspectiva del restaurador local y la soberanía tecnológica:

- **Personalización completa y adaptabilidad modular:** A diferencia de las plataformas rígidas, Distrito Gourmet se concibe como un sistema modular que se adapta quirúrgicamente a las necesidades concretas de un restaurante local. Permite la modificación extensiva del panel de administración y de las funcionalidades específicas según los requerimientos operacionales y de flujo de trabajo del negocio en cuestión.
- **Control total y soberanía de los datos del cliente:** El sistema garantiza que los propietarios gestionan la información de reservas y clientes de forma directa. Esto elimina la dependencia de terceros intermediarios, mitiga los riesgos de privacidad y compliance, y permite al negocio construir su propia base de datos de marketing relacional.
- **Modelo de coste justo:** Eliminación de comisiones y suscripciones: La gran ventaja competitiva reside en su modelo de costes. El sistema no impone comisiones por reserva ni requiere suscripciones mensuales. El único coste operativo asociado es el derivado del hosting y el dominio propio del restaurante, lo que lo convierte en una solución económicamente viable y sostenible a largo plazo para pequeños negocios.
- **Diseño intrínsecamente accesible y multilingüe:** La inclusión es un pilar fundamental. El desarrollo contempla desde su concepción el cumplimiento de estándares de accesibilidad (WCAG). Además, ofrece la posibilidad de adaptar la interfaz a diferentes idiomas, incluyendo el valenciano, mejorando la experiencia de uso y la inclusión de todos los usuarios de la comunidad.
- **Potencial de integración futura y evolución estratégica:** El sistema está diseñado con una arquitectura abierta que le permite evolucionar y crecer funcionalmente a la par que el negocio. Se prevé la posibilidad de integrar futuras funcionalidades estratégicas como:
 - Sistemas avanzados de geolocalización para delivery o localización rápida.
 - Módulos de valoraciones y feedback directo de clientes.
 - Estadísticas de ocupación y rendimiento predictivo del negocio.
 - Herramientas robustas para el cumplimiento integral del Reglamento General de Protección de Datos (RGPD)

3. Estudio de viabilidad.

3.1. Análisis DAFO detallado del proyecto

El desarrollo de un sistema de gestión y reservas propio para restaurantes presenta un perfil estratégico bien definido, articulado en las siguientes dimensiones:

Fortalezas

- **Alta personalización y adaptabilidad:** El proyecto, al ser desarrollado a medida, permite una adaptación total a los procesos operativos, la marca y las necesidades específicas de cada restaurante (gestión de menús dinámicos, tipos de mesa, franjas horarias específicas, etc.). Esto contrasta con las plataformas estándar que imponen una estructura rígida.
- **Bajo coste operacional y de mantenimiento:** Una vez completado el desarrollo inicial, el restaurante incurre únicamente en los costes mínimos de hosting y dominio. Esto elimina las elevadas comisiones por reserva o transacción que imponen las plataformas de terceros, haciendo el sistema significativamente más rentable a largo plazo, especialmente para negocios con un alto volumen de clientes.
- **Dominio de tecnologías estándar y maduras:** El uso de la pila LAMP (Linux, Apache, MySQL, PHP) junto con HTML, CSS y JavaScript garantiza la interoperabilidad, la facilidad de encontrar soporte y la estabilidad del sistema. Son tecnologías probadas y con una vasta comunidad de desarrolladores.
- **Control total de datos y autonomía:** El restaurante mantiene la propiedad y el control exclusivo sobre los datos de sus clientes (CRM, historiales de reserva, preferencias). Esto no solo cumple con las regulaciones de privacidad (GDPR/LOPD), sino que también permite estrategias de marketing y fidelización más efectivas sin dependencia de intermediarios.

Debilidades

- **Dependencia de un desarrollador único:** Al ser un proyecto individual, la limitación de recursos humanos impacta directamente en la velocidad de desarrollo y la capacidad de respuesta ante errores o nuevas peticiones. La escalabilidad y la gestión de la deuda técnica son desafíos constantes.
- **Enfoque en producto mínimo viable (MVP):** Debido al calendario académico, se ha priorizado la robustez de las funciones core (motor de reservas y catálogo

dinámico) sobre funcionalidades secundarias. Esto garantiza que la versión inicial sea técnica y operativamente estable para su uso real desde el primer día.

- **Gestión manual de infraestructura (DevOps):** El proceso de despliegue, configuración del hosting, gestión de bases de datos y mantenimiento del servidor debe ser realizado manualmente. Esto requiere conocimientos técnicos de DevOps que podrían ser un obstáculo si el proyecto se transfiere a un cliente sin soporte técnico.

Oportunidades

- **Aceleración de la digitalización del sector local:** Existe una tendencia clara, impulsada por la experiencia postpandemia, donde los pequeños y medianos restaurantes buscan activamente soluciones digitales para mejorar la eficiencia y la experiencia del cliente.
- **Demanda de soluciones "libres de comisiones":** El alto coste de las plataformas consolidadas ha generado un nicho de mercado para soluciones propietarias y económicas que ofrecen las mismas funcionalidades sin la carga financiera de las comisiones por servicio.
- **Potencial de crecimiento a redes y franquicias:** Una vez probado y validado el sistema en un restaurante local, existe una oportunidad significativa para paquetizar el producto y ofrecerlo como una solución estandarizada a pequeñas cadenas o franquicias gastronómicas.

Amenazas

- **Fuerte competencia de plataformas líderes:** La presencia de gigantes como TheFork, OpenTable, o incluso sistemas integrados en TPVs (Terminales Punto de Venta) ya conocidos y con presupuestos de marketing masivos, representa un desafío considerable para la captación inicial de usuarios.
- **Evolución acelerada del entorno tecnológico web:** El rápido avance de frameworks de JavaScript (React, Vue, Angular) y las nuevas versiones de PHP pueden hacer que las decisiones tecnológicas tomadas al inicio queden obsoletas antes de lo previsto, exigiendo una constante actualización.
- **Dificultad en la adopción inicial:** Persuadir a los restaurantes y, más importante, a los clientes finales, para que utilicen un sistema nuevo en lugar de las soluciones ya establecidas y familiares (ej. Google Maps + OpenTable) requiere una

experiencia de usuario excepcionalmente fluida.

3.2. Estudio de mercado y viabilidad

El contexto actual del mercado de la restauración exige una solución que equilibre la funcionalidad avanzada con la economía y la independencia. Justificación del producto y posicionamiento.

El sector de la restauración ha evolucionado hacia una experiencia híbrida: el cliente espera la inmediatez y la comodidad de la digitalización para tareas como la reserva de mesa, la consulta de la carta o la realización de un pedido takeaway o delivery.

- **Necesidad del producto:** El producto es imperativo porque responde a una carencia específica en el mercado: sistemas robustos de gestión digital que no penalicen al restaurante con un porcentaje de sus ingresos. Muchos restaurantes, especialmente los de alta cocina o los pequeños negocios con márgenes ajustados, prefieren una inversión inicial fija a un coste variable que erosiona su rentabilidad.
- **Hueco de mercado:** El nicho de mercado son los restaurantes de gestión familiar, los locales singulares y los pequeños grupos gastronómicos que valoran la imagen de marca, la relación directa con el cliente y la optimización de costes. Este sistema es una alternativa directa a la dependencia de agregadores.

Público objetivo

- **Restaurantes independientes y pequeños negocios gastronómicos:** Aquellos que necesitan una solución completa de gestión interna (reservas, mesas, pedidos) sin pagar comisiones elevadas.
- **Clientes finales (Diners):** Usuarios de tecnología que buscan un proceso de reserva o pedido online directo, rápido y sin la sobrecarga de información o publicidad de las grandes plataformas.
- **Propietarios/Managers de restaurantes:** Individuos que buscan herramientas internas para obtener una visión analítica completa de sus operaciones y datos de clientes, facilitando la toma de decisiones informadas.

3.3. Viabilidad técnica y económica del proyecto

3.3.1. Viabilidad técnica

Hardware:

- **Recursos detallados:** Ordenador personal de desarrollo. Servidor de pruebas local (XAMPP). Hosting y dominio comercial estándar (ej. SiteGround, Raiola, etc.)
- **Observaciones:** La inversión inicial en hardware es nula. El hosting básico es suficiente para una versión inicial.

Software y stack:

- **Recursos detallados:** Lenguajes: HTML5, CSS3, JavaScript (ES6+), PHP (versión 8.x recomendada), SQL. Frameworks/Librerías: Bootstrap (para diseño responsive rápido), Laravel (backend MVC) o Vue.js (frontend SPA/reactividad). Herramientas: VS Code (IDE), phpMyAdmin/MySQL Workbench, Git/GitHub (control de versiones), Figma (prototipado UI/UX).
- **Observaciones:** Se utilizarán herramientas open source o gratuitas. La posible inclusión de frameworks modernos aumenta la robustez del código y la mantenibilidad.

Recursos Humanos:

- **Recursos detallados:** 1 desarrollador principal (el alumno). Soporte y tutoría técnica del profesorado para validación de arquitectura y mejores prácticas.
- **Observaciones:** La eficiencia del desarrollo depende de una planificación rigurosa y la minimización del scope.

3.3.2. Viabilidad económica

El proyecto presenta una alta viabilidad económica debido a su estructura de costes:

- **Costes fijos:** El único coste recurrente es la adquisición y renovación del dominio y el servicio de hosting. El rango de 50 € a 100 € anuales es manejable y predecible.
- **Costes variables:** Prácticamente nulos, ya que no se utilizan licencias de software propietario ni servicios cloud costosos para la versión inicial.
- **Modelo de negocio potencial:** Este bajo coste permite ofrecer el sistema a un

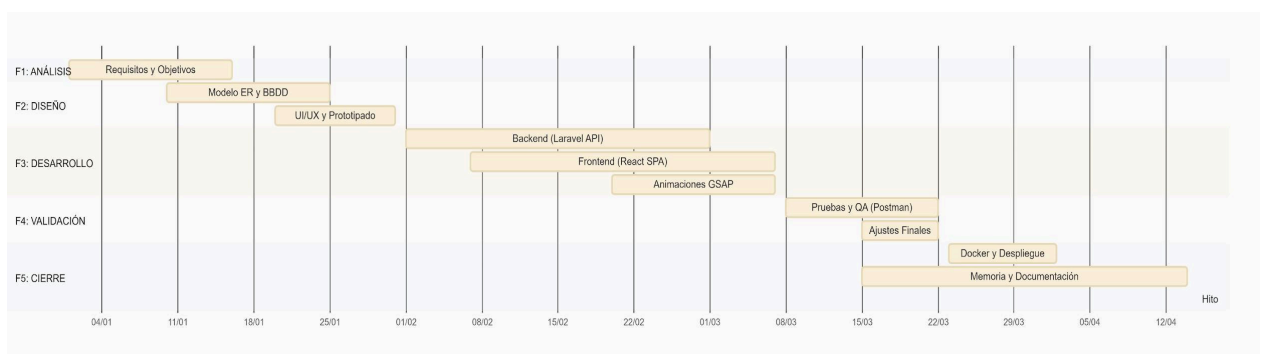
precio competitivo (licencia anual o pago único por instalación) o incluirlo como un servicio de valor añadido en un paquete de consultoría digital.

3.3.3. Viabilidad temporal

Para la organización del desarrollo del proyecto, se ha definido una planificación temporal estructurada en distintas fases. Cada una de ellas agrupa tareas específicas y cuenta con una duración estimada, lo que permite llevar un control progresivo del desarrollo del sistema y asegurar el cumplimiento de los objetivos establecidos.

Esta planificación sigue un enfoque secuencial, en el que cada fase depende en mayor o menor medida de la anterior, facilitando así una correcta evolución del proyecto desde su concepción hasta su despliegue final.

A continuación, se muestra el diagrama de Gantt correspondiente:



Descripción de las fases

- **Análisis y planificación:** En esta fase inicial se definen los requisitos funcionales y no funcionales del sistema, se diseña la base de datos mediante un diagrama entidad-relación y se seleccionan las tecnologías a utilizar durante el desarrollo.
- **Diseño del sistema (UI/UX):** Se realiza el diseño de la interfaz de usuario mediante prototipos de baja y alta fidelidad, así como la definición de la arquitectura del proyecto y la organización de sus componentes.
- **Desarrollo backend:** Se implementa la lógica del servidor, incluyendo la creación de la base de datos, las rutas o APIs necesarias y el sistema de autenticación y control de acceso.
- **Desarrollo frontend:** Se desarrolla la interfaz de usuario adaptativa, integrando los formularios, validaciones y la comunicación con el backend mediante peticiones

HTTP.

- **Pruebas y validación:** Se realizan pruebas funcionales, de integración y de usabilidad con el objetivo de detectar errores y garantizar el correcto funcionamiento del sistema.
- **Despliegue y documentación:** Se lleva a cabo la publicación del proyecto en un entorno de producción, junto con la configuración final del sistema y la elaboración de la documentación técnica y de usuario.

Dependencias del proyecto

El desarrollo del proyecto sigue una estructura lógica en la que las distintas fases están interrelacionadas:

- El análisis y la planificación constituyen la base de todo el proyecto.
- El diseño del sistema depende directamente de los requisitos definidos previamente.
- El desarrollo backend se apoya en el diseño de la base de datos y la arquitectura establecida.
- El frontend depende de la disponibilidad de las funcionalidades del backend.
- Las pruebas requieren que tanto el backend como el frontend estén implementados.
- El despliegue final solo puede realizarse una vez validado el correcto funcionamiento del sistema.

4. Análisis de requisitos

4.1. Descripción de requisitos

4.1.1. Texto explicativo

El sistema web Distrito Gourmet tiene como objetivo ofrecer una plataforma propia para la gestión de reservas y pedidos online de un restaurante. A continuación, se detallan los requisitos funcionales a nivel frontend, backend y base de datos:

Frontend (interfaz de usuario):

- **Página de inicio/acceso:** Presenta información básica del restaurante (nombre, horarios, dirección, contacto) y permite al usuario acceder a las secciones de reserva o pedido.
- **Formulario de inicio de sesión y registro:** Permite a los clientes registrarse con sus datos personales (nombre, correo, contraseña) y acceder al sistema para gestionar sus reservas. El administrador tiene acceso diferenciado al panel de gestión.
- **Motor de Reservas Inteligente:** Permite la selección de fecha y hora con validación de disponibilidad en tiempo real. El sistema está vinculado a la capacidad de sala configurada en el backend, evitando duplicidades y permitiendo al usuario preseleccionar menús específicos.
- **Confirmación y modificación de reservas:** Los usuarios pueden visualizar el estado de sus reservas, modificarlas o cancelarlas, recibiendo notificaciones por correo electrónico.
- **Catálogo Gastronómico Dinámico:** Interfaz para la gestión de pedidos online que permite la visualización detallada de platos, ingredientes y alérgenos. Incluye una lógica de carrito de compra optimizada para procesos de recogida (takeaway) o entrega.
- **Diseño responsive y accesible:** La interfaz se adapta a distintos tamaños de pantalla (ordenador, móvil o tablet) y cumple los estándares de accesibilidad WCAG.

Backend (lógica del servidor):

- **Gestión de usuarios:** Autenticación y autorización de clientes y administradores. Los administradores pueden visualizar, modificar o eliminar reservas.
- **Gestión de reservas:** Creación, modificación y eliminación de reservas, validando

que no haya duplicidad de horario o mesa.

- **Gestión de pedidos:** Registro, actualización de estado (pendiente, en preparación, entregado) y notificación al cliente.
- **Panel de administración:** Permite al propietario visualizar estadísticas de ocupación, lista de reservas del día, historial de clientes y configuración de horarios del restaurante.
- **Envío de correos electrónicos:** El sistema queda preparado para el envío de confirmaciones mediante el sistema de correo de Laravel, configurable por SMTP desde el archivo .env.
- **Seguridad:** Cifrado de contraseñas (bcrypt o password_hash), validación de formularios y protección contra ataques de inyección SQL.

Base de datos (BBDD): La base de datos se implementará en MySQL, y contendrá tablas principales como:

- **usuarios** (id, nombre, correo, contraseña, rol)
- **reservas** (id, usuario_id, fecha, hora, comensales, estado)
- **pedidos** (id, usuario_id, fecha, hora, total, estado)
- **platos** (id, nombre, descripción, precio, categoría, imagen)
- **configuración** (id, horario_apertura, horario_cierre, mesas_disponibles)

Todas las tablas estarán relacionadas mediante claves foráneas y cumplirán con la 3.^a forma normal para garantizar la integridad de los datos.

Objetivos globales del sistema:

- Digitalizar y optimizar el proceso de reservas y pedidos.
- Mejorar la experiencia del cliente mediante un entorno web intuitivo.
- Reducir errores y tiempos de gestión manual.
- Ofrecer independencia tecnológica y control de los datos al restaurante.

4.1.2. Diagramas de caso de uso

A continuación, se describen los casos de uso más relevantes para la aplicación, atendiendo a los distintos perfiles (cliente, administrador, sistema):

Caso de uso general — Distrito Gourmet

- Actores: Cliente, Administrador, Sistema.

- Casos de uso principales:
 - Cliente: registrarse, iniciar sesión, realizar reserva, modificar o cancelar reserva, realizar pedido.
 - Administrador: gestionar reservas, visualizar estadísticas, configurar horarios y menú.
 - Sistema: validar disponibilidad, enviar correos, actualizar base de datos.

Caso de uso específico — Reserva de mesa

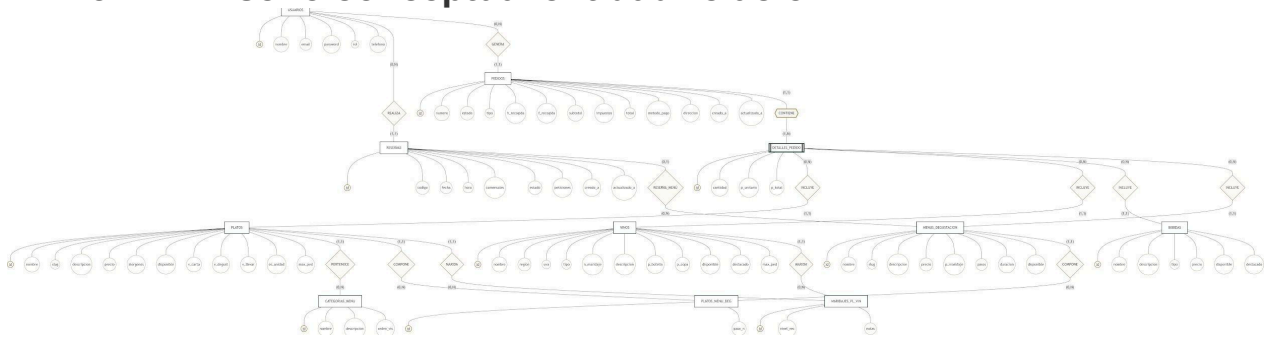
- Actor principal: Cliente.
- Flujo básico:
 1. El cliente accede al formulario de reserva.
 2. Selecciona fecha, hora y número de comensales.
 3. El sistema comprueba disponibilidad.
 4. Si hay mesa libre, registra la reserva. El cliente recibe una confirmación por correo.

Caso de uso específico — Pedido online

- Actor principal: Cliente.
- Flujo básico:
 1. El cliente accede al menú digital.
 2. Selecciona los platos y confirma el pedido.
 3. El sistema calcula el importe total y lo registra.
 4. El administrador visualiza el pedido en el panel.
 5. El sistema actualiza el estado y envía notificación.

5. Diseño

5.1. Diseño conceptual entidad-relación



Debido a las dimensiones del diagrama, la imagen ha sido reducida para su maquetación. [\[Haga clic aquí\]](#)

El diagrama entidad-relación representa la estructura conceptual de la base de datos de Distrito Gourmet. En él se identifican las entidades principales del sistema, como usuarios, reservas, pedidos, platos, bebidas, vinos, menús de degustación y detalles de pedido.

La entidad usuarios actúa como núcleo de la aplicación, ya que se relaciona con las reservas y los pedidos realizados por cada cliente. Las reservas pueden asociarse opcionalmente a un menú de degustación, mientras que los pedidos se descomponen en líneas mediante la entidad detalles_pedido, permitiendo incluir platos, bebidas, vinos o menús.

El modelo también contempla relaciones específicas del dominio gastronómico, como la agrupación de platos por categorías, la composición de menús de degustación mediante platos_menu_degustacion y las recomendaciones de maridaje entre platos y vinos. Este diseño permite mantener la información normalizada, evitar duplicidades y facilitar futuras ampliaciones del sistema.

5.2. Diseño lógico relacional.

A partir del modelo conceptual, se ha realizado el proceso de normalización hasta (3FN), garantizando que cada atributo dependa únicamente de la clave primaria. Notación:

Negrita: Clave primaria (Primary Key - PK).

Cursiva con asterisco ():* Clave foránea (Foreign Key - FK).

- **USUARIOS** (**id**, nombre, email, password, rol, telefono)
- **CATEGORIAS_MENU** (**id**, nombre, descripcion, orden_visualizacion)
- **PLATOS** (**id**, nombre, slug, descripcion, precio, *categoria_menu_id**, alergenos,

disponible, visible_en_carta, visible_en_degustacion, disponible_para_llevar, es_por_unidad, maximo_por_pedido)

- **VINOS** (id, nombre, region, uva, tipo, notas_maridaje, descripcion, precio_botella, precio_copa, disponible, destacado, maximo_por_pedido)
- **BEBIDAS** (id, nombre, descripcion, tipo, precio, disponible, destacado)
- **MENUS_DEGUSTACION** (id, nombre, slug, descripcion, precio, precio_maridaje, pasos, duracion_estimada_minutos, disponible)
- **PLATOS_MENU_DEGUSTACION** (id, menu_degustacion_id*, plato_id*, numero_paso)
- **MARIDAJES_PLATO_VINO** (id, plato_id*, vino_id*, nivel_recomendacion, notas)
- **RESERVAS** (id, usuario_id*, menu_degustacion_id*, codigo_reserva, fecha_reserva, hora_reserva, comensales, estado, peticiones_especiales, creado_a, actualizado_a)
- **PEDIDOS** (id, usuario_id*, numero_pedido, estado, tipo_pedido, hora_recogida, fecha_recogida, subtotal, impuestos, total, metodo_pago, direccion, creado_a, actualizado_a)
- **DETALLES_PEDIDO** (id, pedido_id*, plato_id*, vino_id*, bebida_id*, menu_degustacion_id*, cantidad, precio_unitario, precio_total)

5.3. Diseño físico o diagrama MySQL

Implementación técnica sobre MySQL 8.0 / InnoDB.

Tabla: usuarios

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID único auto-incremental.
nombre	VARCHAR(255)	NO		Nombre completo.
email	VARCHAR(255)	NO	UNI	Login / Correo de contacto.
password	VARCHAR(255)	NO		Hash Bcrypt.
rol	ENUM	NO		'Administrador', 'Staff', 'Cliente'.
telefono	VARCHAR(255)	SÍ		Contacto telefónico.

Tabla: categorias_menu

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de la categoría.
nombre	VARCHAR(255)	NO		Título (ej: 'Entrantes').
descripcion	TEXT	SÍ		Detalle de la sección.
orden_visualizacion	INT	NO		Prioridad en la carta (0-99).

Tabla: platos

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID único del plato.
nombre	VARCHAR(255)	NO		Nombre comercial.
slug	VARCHAR(255)	NO	UNI	Identificador URL amigable.
descripcion	TEXT	SÍ		Ingredientes y preparación.
precio	DECIMAL(10,2)	NO		Precio base P.V.P.
categoria_menu_id	BIGINT UNSIGNED	SÍ	FK	Referencia a categorias_menu.
alergenos	VARCHAR(255)	SÍ		Lista de iconos de alérgenos.
disponible	TINYINT(1)	NO		Stock disponible (0/1).
visible_en_carta	TINYINT(1)	NO		Mostrar en carta pública.
visible_en_degustacion	TINYINT(1)	NO		Apto para degustación.
disponible_para_llevar	TINYINT(1)	NO		Apto para Takeaway.
es_por_unidad	TINYINT(1)	NO		Precio por pieza o ración.
maximo_por_pedido	INT	NO		Límite por cliente.

Tabla: vinos

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de la botella.
nombre	VARCHAR(255)	NO		Marca y añada.
region	VARCHAR(255)	SÍ		D.O. (Denominación Origen).
uva	VARCHAR(255)	SÍ		Variedad de uva.
tipo	ENUM	NO		'Tinto', 'Blanco', 'Rosado', etc.
notas_maridaje	TEXT	SÍ		Recomendaciones del sumiller.
precio_botella	DECIMAL(10,2)	SÍ		Precio formato botella.
precio_copa	DECIMAL(10,2)	SÍ		Precio formato copa.
destacado	TINYINT(1)	NO		Recomendación de la semana.

Tabla: bebidas

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de la bebida.
nombre	VARCHAR(255)	NO		Nombre comercial.
tipo	ENUM	NO		'agua', 'refresco', 'cocktail', 'cafe'.
precio	DECIMAL(10,2)	NO		Precio de venta.

Tabla: menus_degustacion

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID del menú.
nombre	VARCHAR(255)	NO		Título del menú.
precio	DECIMAL(10,2)	NO		Precio por comensal.
precio_maridaje	DECIMAL(10,2)	SÍ		Suplemento opcional bodega.
pasos	INT	NO		Nº de platos que lo componen.
duracion_estimada_minutos	INT	SÍ		Tiempo de servicio previsto.

Tabla: platos_menu_degustacion

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de la relación.
menu_degustacion_id	BIGINT UNSIGNED	NO	FK	ID del menú asociado.
plato_id	BIGINT UNSIGNED	NO	FK	ID del plato asociado.
numero_paso	INT	NO		Orden del plato en el menú.

Tabla: maridajes_plato_vino

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de la relación.
plato_id	BIGINT UNSIGNED	NO	FK	ID del plato.
vino_id	BIGINT UNSIGNED	NO	FK	ID del vino sugerido.
nivel_recomendacion	ENUM	NO		'Buena', 'Muy buena', 'Perfecta'.

Tabla: reservas

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de reserva.
usuario_id	BIGINT UNSIGNED	NO	FK	ID del cliente.
codigo_reserva	VARCHAR(255)	SÍ	UNI	Localizador único.
fecha_reserva	DATE	SÍ		Fecha prevista.
hora_reserva	TIME	SÍ		Hora del turno.
comensales	INT	NO		Nº de personas.
menu_degustacion_id	BIGINT UNSIGNED	SÍ	FK	Menú pre-seleccionado.
estado	VARCHAR(255)	NO		'Pendiente', 'Confirmada', etc.
creado_a	TIMESTAMP	SÍ		Timestamp de creación.

Tabla: pedidos

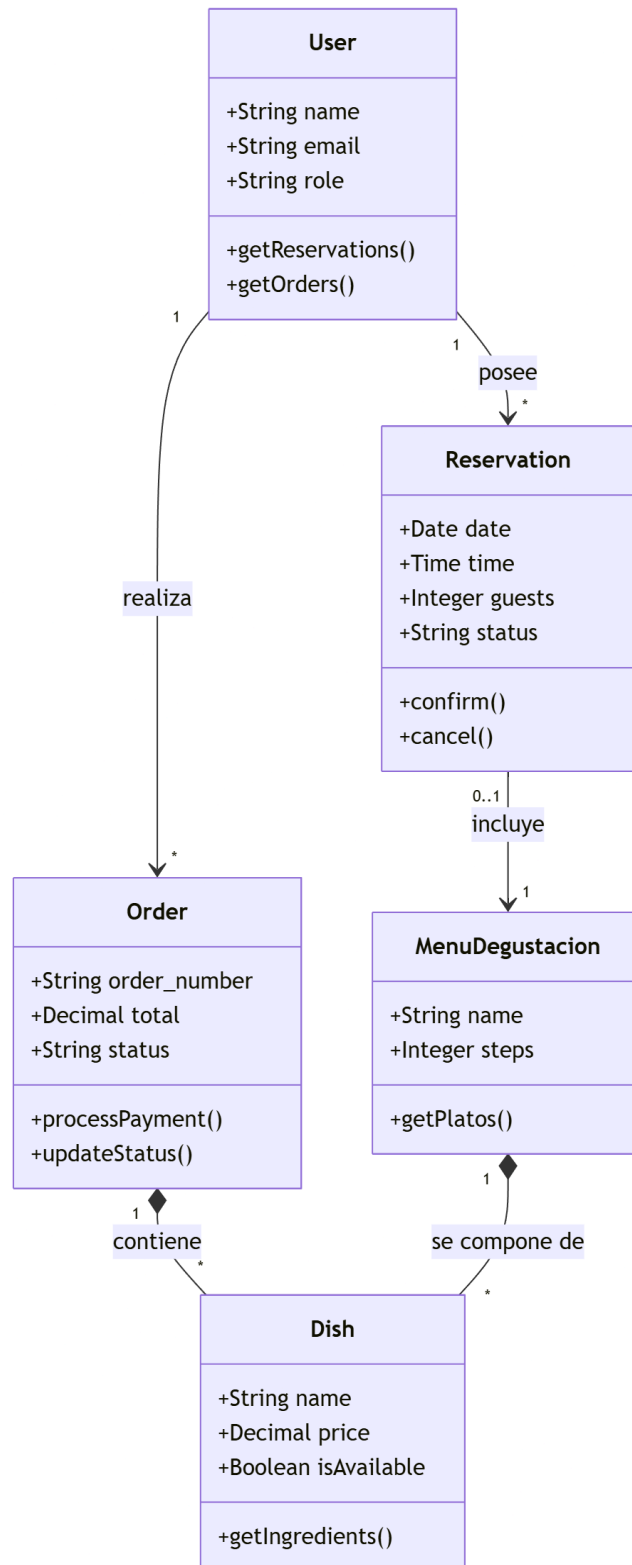
Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de pedido.
numero_pedido	VARCHAR(255)	SÍ	UNI	Localizador de comanda.
estado	ENUM	NO		'Pendiente', 'Preparando', etc.
tipo_pedido	ENUM	NO		'Sala', 'Takeaway', 'Delivery'.
hora_recogida	TIME	SÍ		Hora para Takeaway.
fecha_recogida	DATE	SÍ		Fecha para Takeaway.
subtotal	DECIMAL(10,2)	NO		Base imponible.
impuestos	DECIMAL(10,2)	NO		IVA calculado.
total	DECIMAL(10,2)	NO		Importe final.
metodo_pago	VARCHAR(255)	SÍ		Ej: 'Tarjeta'.

Tabla: detalles_pedido

Campo	Tipo	Nulidad	Key	Descripción
id	BIGINT UNSIGNED	NO	PK	ID de línea.
pedido_id	BIGINT UNSIGNED	NO	FK	Cabecera del pedido.
plato_id	BIGINT UNSIGNED	SÍ	FK	Plato incluido (opcional).
vino_id	BIGINT UNSIGNED	SÍ		Vino incluido (opcional).
bebida_id	BIGINT UNSIGNED	SÍ		Bebida incluida (opcional).
menu_degustacion_id	BIGINT UNSIGNED	SÍ	FK	Menú incluido (opcional).
cantidad	INT	NO		Unidades.
precio_unitario	DECIMAL(10,2)	SÍ		PVP histórico línea.
precio_total	DECIMAL(10,2)	SÍ		Subtotal línea.

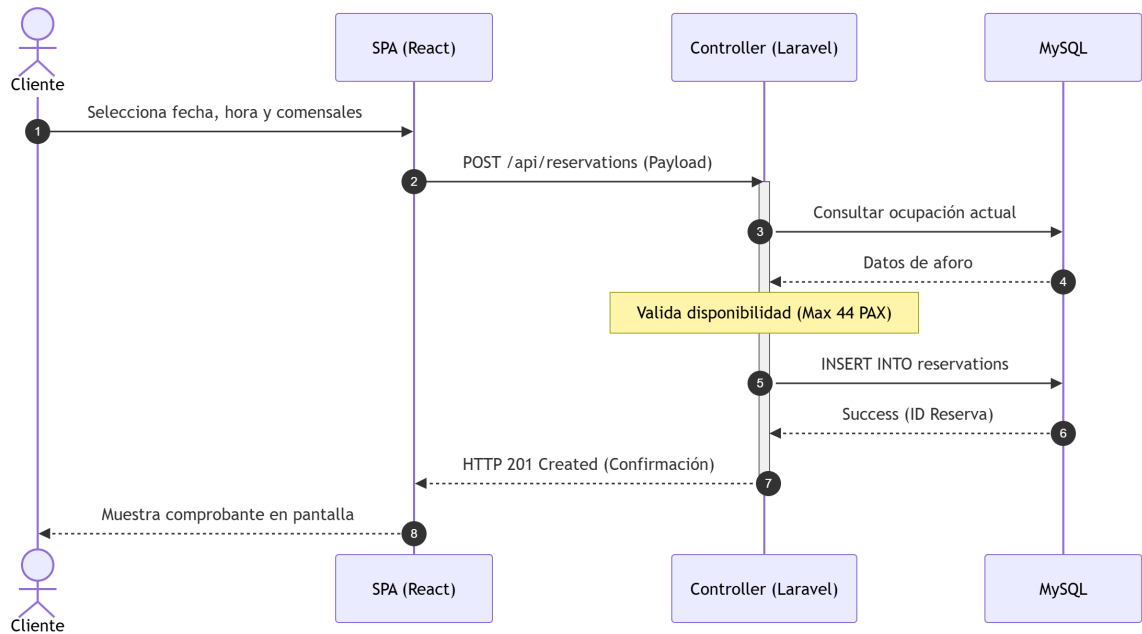
5.4. Orientación a objetos

5.4.1. Diagrama de clases



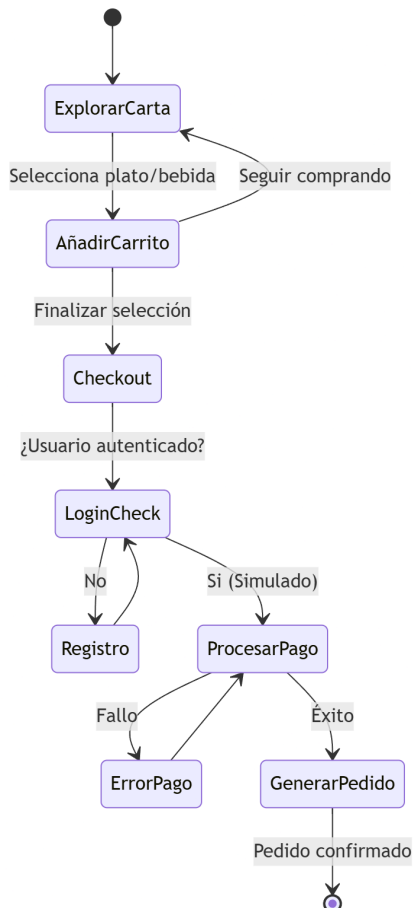
El diagrama de clases muestra las entidades principales del sistema y sus relaciones desde el punto de vista de la lógica de aplicación.

5.4.2. Diagrama de secuencias.



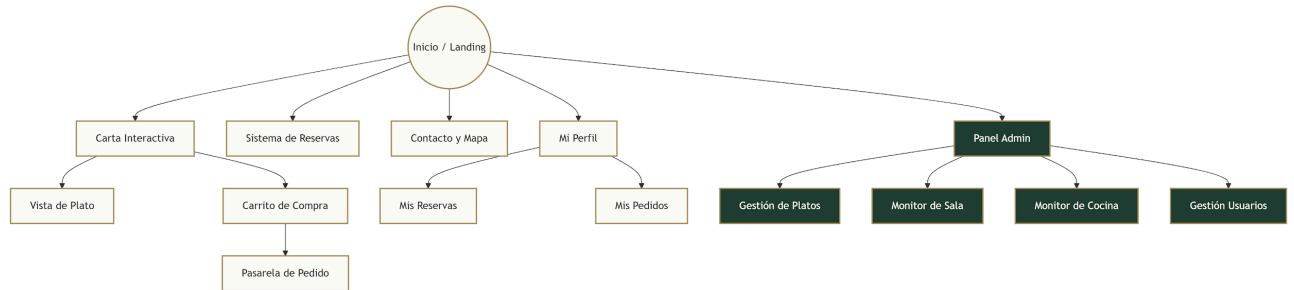
El diagrama de secuencia representa el flujo principal de interacción entre cliente, frontend, API y base de datos durante el proceso de reserva.

5.4.3. Diagrama de actividad



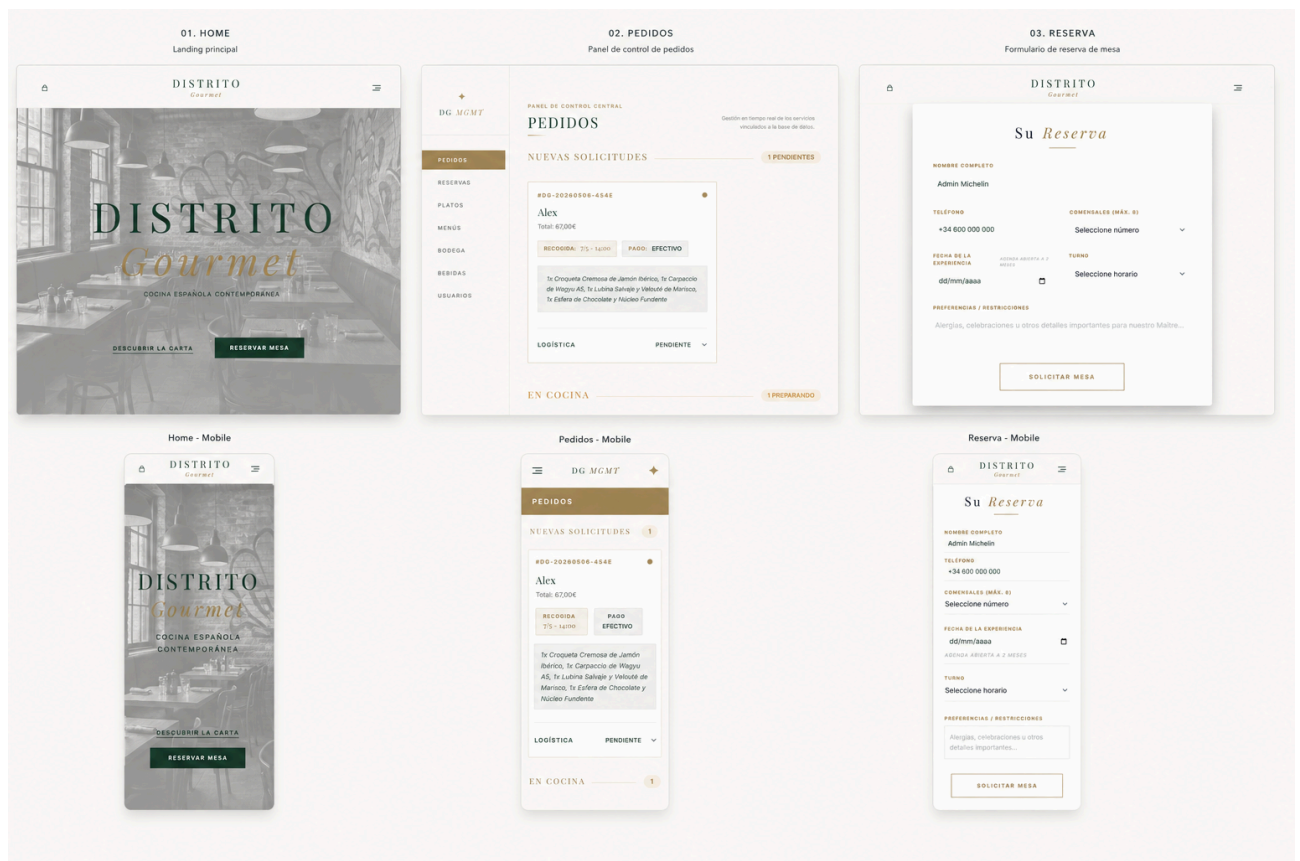
El diagrama de actividad describe el comportamiento funcional del sistema desde que el usuario inicia una acción hasta que recibe confirmación.

5.4.4. Mapa web



El mapa web recoge la estructura de navegación de la aplicación y las principales secciones accesibles para cliente y administrador.

5.4.5. Mockups



Los mockups muestran la propuesta visual inicial de la interfaz y sirven como referencia para el desarrollo del frontend.

6. Codificación

6.1. Tecnologías elegidas y su justificación

Para el desarrollo de Distrito Gourmet, se ha seleccionado un stack tecnológico moderno que permite una separación total entre el servidor y el cliente.

- **Backend:** Laravel 12 (PHP 8.2+): Se ha elegido Laravel por ser un framework robusto que me permite gestionar la lógica de negocio de forma eficiente. Su ORM Eloquent facilita la gestión de la base de datos de manera intuitiva y segura.
- **Frontend:** React 19 + Vite: Se ha optado por React 19 por su potencia y rendimiento. Se utiliza Vite como entorno de desarrollo por su rapidez en desarrollo y compilación, lo que optimiza mucho los tiempos de compilación.
- **Estado global (Zustand):** Se ha utilizado Zustand para gestionar el estado de la aplicación (carrito y sesión). Es una opción mucho más ligera que Redux y se integra perfectamente con React 19.
- **Navegación (React Router 7):** Se utiliza React Router para gestionar las rutas de la SPA, permitiendo una navegación fluida entre la carta, las reservas y el panel de administración.
- **Animaciones (GSAP):** Se ha integrado GSAP para crear microinteracciones y efectos de scroll que refuerzan la identidad visual del proyecto.
- **Base de datos (MySQL):** Es el motor principal por su fiabilidad. Se ha desarrollado el seeder DistritoGourmetSeeder para que la base de datos cuente con un catálogo gastronómico real (fotos, descripciones, precios) desde el primer momento.

Comparación y justificación tecnológica.

Frente a opciones como Node.js, Laravel ofrece una estructura de proyecto más sólida y herramientas integradas para la seguridad y las migraciones. En el frontend, React 19 es la opción más flexible para crear componentes reutilizables y gestionar animaciones complejas con GSAP.

6.2. Entorno servidor

6.2.1. Descripción general

El servidor funciona como una API RESTful. Recibe peticiones HTTP, procesa la lógica en los controladores de Laravel y devuelve respuestas en formato JSON, manteniendo el backend totalmente independiente del cliente.

6.2.2. Seguridad y prevención de inyección SQL

- **Sentencias preparadas:** Se evita la inyección SQL mediante el uso de Eloquent ORM, que utiliza internamente parámetros vinculados (PDO) para todas las consultas.
- **Validación:** Todas las entradas son validadas mediante Request Validation antes de procesar cualquier dato.
- **Tokenización:** Uso de Laravel Sanctum para gestionar sesiones seguras mediante tokens cifrados.

6.2.3. Gestión de errores y warnings

- **Excepciones:** Uso de bloques try-catch en procesos críticos como reservas y pedidos para capturar fallos y devolver mensajes de error controlados.
- **Log System:** Registro de anomalías en los logs del servidor para una depuración proactiva.

6.3. Entorno cliente

6.3.1. Descripción general

La interfaz es una SPA (Single Page Application) construida con React. La comunicación asíncrona con el backend se realiza mediante Axios, permitiendo una navegación instantánea sin recargas de página.

6.3.2. Compatibilidad con navegadores

La aplicación se ha orientado a navegadores modernos como Chrome, Edge, Firefox y Safari. Internet Explorer no se considera objetivo de compatibilidad debido al uso de React 19 y estándares actuales de JavaScript y CSS.

6.4. Documentación interna de código

6.4.1. Esquema de documentación (ejemplo backend)

Cada controlador y función en el entorno servidor sigue el estándar PHPDoc. Ejemplo:

```
</?php
// Gestión integral de las reservas y el control de aforo del restaurante.
namespace App\Http\Controllers\API;
```

6.4.2. Esquema de documentación (ejemplo frontend)

En el entorno cliente, los componentes de React utilizan JSDoc:

```
// Gestión de la carta: carga de datos desde la API y filtrado por categorías.
import { useCartStore } from "@store/cart";
```

6.5. Documentación externa

Para este proyecto, se ha centralizado toda la información técnica y de usuario en archivos Markdown (.md), guardados dentro de la carpeta docs/ en la raíz del repositorio. Esto permite que la documentación esté siempre accesible, versionada junto al código y fácil de actualizar.

6.5.1. Manual del usuario.

Se ha redactado un manual pensado para que cualquier cliente pueda usar la aplicación sin conocimientos técnicos, recogido en el archivo docs/MANUAL_USUARIO.md.

- **Acceso:** Está enlazado desde el pie de página de la aplicación web, permitiendo consultarlo directamente desde el navegador sin salir de la web.
- **Contenido:** Cubre de forma sencilla los cuatro flujos principales: registro/login, navegación por la carta, proceso de pedido takeaway y sistema de reservas, explicando los estados posibles en cada caso (Pendiente, Confirmada, Cancelada, etc.).

6.5.2. Documentación de la API

Se han documentado todos los endpoints de la API en el archivo docs/API_DOCS.md, con el objetivo de que cualquier desarrollador pueda integrarse con el sistema sin necesidad de revisar el código fuente.

- **Contenido:** Incluye la URL base, el método de autenticación (Bearer Token vía Sanctum), ejemplos de petición y respuesta en formato JSON para cada endpoint, y una tabla de códigos de error.
- **Rutas documentadas:** Autenticación, carta pública, reservas, pedidos y panel de administración completo.

6.5.3. Manual de despliegue y mantenimiento.

El archivo docs/DEPLOY.md contiene las instrucciones necesarias para poner en marcha el proyecto tanto en local como en producción.

- **Despliegue local:** Describe el proceso completo desde clonar el repositorio hasta arrancar frontend y backend con el comando `npm start`, incluyendo la configuración del `.env` y la ejecución del `DistritoGourmetSeeder`.
- **Despliegue con Docker:** Explica cómo levantar todos los servicios (MySQL, Laravel, React, Nginx) con un único comando `docker-compose up -d --build`

6.6. Ejemplos de implementación

En este apartado se muestran algunos fragmentos representativos del proyecto. Se han seleccionado solo las partes más relevantes para explicar la estructura general de la aplicación, la comunicación entre frontend y backend y la relación entre los modelos de datos.

6.6.1. Rutas de la API

Este fragmento muestra cómo se expone una funcionalidad concreta del backend mediante una ruta de Laravel. Es importante porque define el punto de entrada desde el que el frontend puede interactuar con el servidor.

```
php
Route::post('/reservations', [ReservaController::class, 'store']);
```

Esta ruta permite crear reservas desde la interfaz de usuario y conecta directamente el formulario del frontend con la lógica del backend.

6.6.2. Lógica del backend

Este fragmento corresponde al controlador encargado de procesar una reserva. Aquí se validan los datos recibidos y se aplica la lógica necesaria antes de guardar la información en la base de datos.

php

```

public function store(Request $request)
{
    $request->validate([
        'fecha_reserva' => 'required|date',
        'hora_reserva' => 'required|string',
        'comensales' => 'required|integer|min:1',
        'peticiones_especiales' => 'nullable|string'
    ]);
    $exists = Reserva::where('usuario_id', auth()->id())
        ->where('fecha_reserva', $request->fecha_reserva)
        ->where('estado', '!=', 'Cancelada')
        ->exists();
    if ($exists) {
        return response()->json([
            'mensaje' => 'Ya dispone de una reserva para esta fecha.'
        ], 422);
    }
    $res = Reserva::create([
        'usuario_id' => auth()->id(),
        'fecha_reserva' => $request->fecha_reserva,
        'hora_reserva' => $request->hora_reserva,
        'comensales' => $request->comensales,
        'peticiones_especiales' => $request->peticiones_especiales,
        'codigo_reserva' => strtoupper(substr(uniqid(), -8))
    ]);
    return response()->json([
        'mensaje' => 'Reserva creada correctamente',
        'reserva' => $res
    ], 201);
}

```

Este ejemplo es relevante porque resume el funcionamiento principal del backend en una operación real: validación de datos, comprobación de reglas de negocio y creación del registro en la base de datos.

6.6.3. Modelo y relación

Este fragmento muestra cómo se relaciona una reserva con el usuario que la ha realizado. La relación entre modelos es importante porque permite estructurar correctamente la base de datos y acceder a los datos de forma más sencilla.

php

```

public function usuario() {
    return $this->belongsTo(Usuario::class, 'usuario_id');
}

```

Gracias a esta relación, cada reserva puede asociarse al usuario correspondiente dentro del sistema.

6.6.4. Consumo de la API desde el frontend

Este fragmento muestra cómo el frontend envía los datos del formulario al backend mediante una petición HTTP. Es uno de los ejemplos más importantes porque refleja la comunicación real entre React y Laravel.

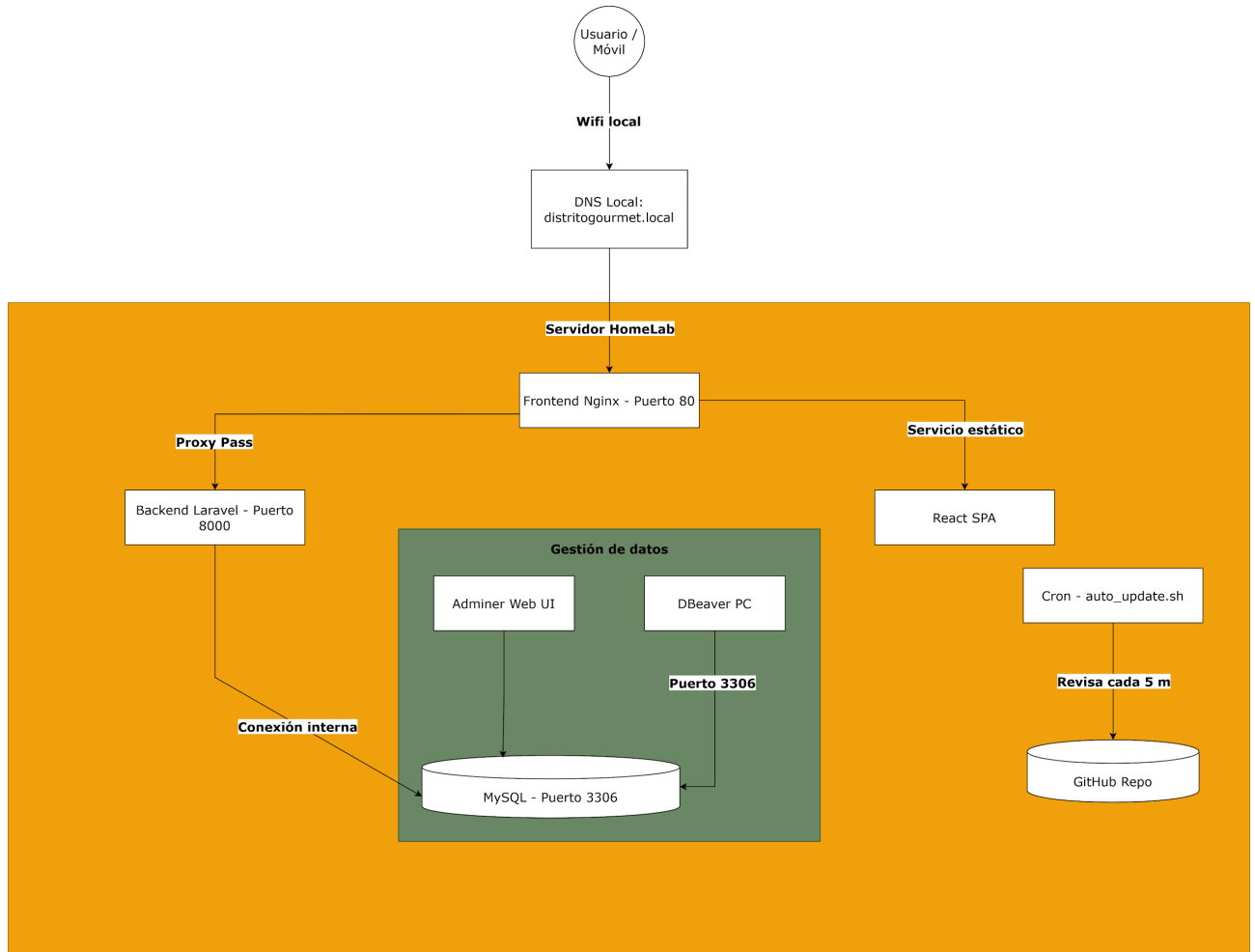
```
jsx
await axios.post("/reservations", {
  nombre: form.name,
  telefono: form.phone,
  fecha_reserva: form.date,
  hora_reserva:
    form.time.includes(":") && form.time.split(":").length === 2
    ? `${form.time}:00`
    : form.time,
  comensales: parseInt(form.guests),
  peticiones_especiales: form.comments,
});
```

Este envío de datos permite registrar la reserva desde la interfaz del usuario y completar el flujo funcional de la aplicación.

En conjunto, estos fragmentos resumen la parte más importante de la implementación: rutas, lógica de negocio, relaciones entre entidades y comunicación entre cliente y servidor.

7. Despliegue

7.1. Diagramas de despliegue



7.2. Descripción de la instalación o despliegue

7.2.1. Fichero de configuración:

Para el correcto funcionamiento, se han configurado los siguientes ficheros clave:

- **Variables de entorno (.env):**
 - **Backend:** Configura la conexión a la base de datos (DB_CONNECTION=mysql), las credenciales y la URL de producción para evitar errores de CORS.
 - **Frontend:** Define la variable VITE_API_URL que apunta a la IP o nombre de host del servidor local para las peticiones de Axios.
- **Orquestación (docker-compose.yml):** Define los tres servicios principales del sistema: base de datos MySQL, backend Laravel y frontend React servido

mediante Nginx. También configura los puertos expuestos, las variables de entorno necesarias y el volumen persistente de MySQL.

- **Servidor web (frontend/nginx.conf):** Configurado para actuar como servidor de archivos estáticos y como reverse proxy, permitiendo el enrutamiento por nombre de host (distritogourmet.local).
- **Servicio de red (mdns-publish.service):** Fichero de unidad de systemd que automatiza la publicación de nombres de host en la red local mediante el protocolo Avahi/mDNS.

7.2.2. Descripción del servidor hosting utilizado.

El proyecto se ha desplegado en un entorno homelab propio, utilizando un servidor local con Ubuntu Server 24.04 LTS. Esta opción se ha elegido para mantener el control completo sobre la infraestructura, los datos de la aplicación y el proceso de despliegue, evitando depender de servicios de hosting externos.

La aplicación se ejecuta mediante Docker y Docker Compose, separando los servicios principales en contenedores independientes: base de datos MySQL, backend Laravel y frontend React servido mediante Nginx. Esta arquitectura facilita el mantenimiento, permite reconstruir servicios de forma aislada y mejora la portabilidad del proyecto.

El acceso dentro de la red local se realiza mediante nombres de dominio `.local`, publicados con Avahi/mDNS. De este modo, los dispositivos de la red pueden acceder a la aplicación mediante un nombre legible, sin tener que recordar la dirección IP del servidor. Además, se ha configurado Nginx como servidor web del frontend y como punto de entrada para la aplicación.

La base de datos MySQL se mantiene en un volumen persistente de Docker, lo que permite conservar la información aunque se reconstruyan los contenedores. También se ha habilitado el acceso desde herramientas externas de administración, como DBeaver, para facilitar la revisión, mantenimiento y consulta directa de los datos durante el desarrollo y las pruebas. Para automatizar el despliegue, se ha implementado un sistema de actualización continua basado en un script Bash ejecutado periódicamente mediante cron. El servidor consulta el repositorio remoto, detecta cambios en la rama principal y, en caso necesario, actualiza el código y reconstruye los contenedores correspondientes. Esto permite mantener el entorno homelab sincronizado con la versión estable del proyecto.

Elemento	Descripción
Servidor	Homelab propio
Sistema operativo	Ubuntu Server 24.04 LTS
Contenerización	Docker y Docker Compose
Backend	Laravel 12 / PHP 8.2
Frontend	React + Vite servido con Nginx
Base de datos	MySQL 8.0 con volumen persistente
Resolución local	Avahi/mDNS con dominio .local
Administración BD	Acceso externo mediante DBeaver
Automatización	Script Bash programado con cron

8. Herramientas de apoyo

8.1. Control de versiones

Para la gestión del código fuente a lo largo del ciclo de vida del proyecto, se ha utilizado Git como sistema de control de versiones, alojando el repositorio remoto en la plataforma GitHub. Se ha seguido una estrategia de ramificación estructurada, separando el desarrollo de nuevas funcionalidades y la corrección de errores de la rama principal (main). Esto ha permitido mantener un historial de cambios limpio, facilitar el seguimiento del progreso y proporcionar un mecanismo de seguridad para revertir cambios en caso de fallos críticos. Adicionalmente, se ha utilizado Visual Studio Code como entorno de desarrollo integrado (IDE) para la escritura del código.

8.2. Sistemas de integración continua

En lugar de utilizar servicios externos de integración, se ha optado por una solución de Despliegue Continuo (CD) personalizada y robusta, adaptada a la infraestructura de homelab del proyecto. Se ha desarrollado un script de automatización en Bash (auto_update.sh) que reside en el servidor de producción. Este sistema funciona bajo la siguiente lógica:

- **Monitorización:** Mediante una tarea programada (cron job), el servidor consulta el repositorio de GitHub cada 5 minutos.
- **Sincronización:** Si se detectan cambios en la rama principal (main), el script realiza un git pull automático para descargar la última versión estable.
- **Redespliegue:** El script invoca a Docker Compose para reconstruir los contenedores necesarios, asegurando que cualquier cambio en la API o en la interfaz se aplique de forma inmediata y sin intervención manual.

Esta solución garantiza que el entorno de producción esté siempre alineado con el código fuente validado en el repositorio.

8.3. Gestión de pruebas

Para asegurar la estabilidad de Distrito Gourmet, se ha seguido una estrategia de pruebas manuales exhaustivas y validación técnica en las herramientas de desarrollo, asegurando que todos los flujos críticos de negocio funcionen correctamente.

- **Pruebas de la API (backend):** Se han validado todos los endpoints documentados

utilizando Postman. Se ha verificado que las rutas protegidas con Laravel Sanctum rechacen peticiones sin token y que las validaciones de datos (como la comprobación de aforo en las reservas) devuelvan los códigos de error 422 y 500 cuando corresponde.

- **Pruebas de interfaz y estado (frontend):** Utilizando las React Developer Tools, se ha monitorizado el estado global de la aplicación gestionado por Zustand. Se han realizado pruebas de persistencia del carrito de compra y de la sesión del usuario para garantizar que la información no se pierda al recargar la página.
- **Pruebas de usuario (end-to-end):** Se han simulado los flujos completos desde la perspectiva de un cliente real: registro de cuenta, navegación por la carta con filtros de alérgenos, proceso de reserva de mesa y realización de pedidos para llevar.
- **Validación de diseño adaptativo:** Se ha comprobado la visualización de la web en múltiples dispositivos (móvil, tablet y escritorio) utilizando las herramientas de inspección de Chrome, asegurando que la experiencia de usuario sea fluida y que las animaciones de GSAP no afecten al rendimiento en dispositivos con menos recursos.

9. Conclusiones.

9.1. Conclusiones sobre el trabajo realizado

El desarrollo de Distrito Gourmet ha permitido construir una aplicación web funcional orientada a la gestión de un restaurante, integrando reservas, pedidos, carta digital y administración básica desde una arquitectura separada entre frontend y backend. Durante el proyecto se ha trabajado con una API desarrollada en Laravel y una interfaz creada con React y Vite, lo que ha permitido separar la lógica del servidor de la experiencia de usuario. Esta separación facilita el mantenimiento del código y permite que cada parte de la aplicación pueda evolucionar de forma independiente. El resultado final cumple los objetivos principales planteados: permitir al usuario consultar la carta, realizar reservas y gestionar pedidos, mientras que la parte administrativa permite controlar la información principal del sistema. Aunque el proyecto todavía podría ampliarse, la base desarrollada es sólida y representa una solución completa para el alcance definido.

9.2. Conclusiones personales

A nivel personal, este proyecto ha supuesto una oportunidad para aplicar de forma conjunta muchos de los conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Web. No solo se ha trabajado la programación del frontend y del backend, sino también el diseño de la base de datos, la organización del proyecto, la documentación y el despliegue.

Uno de los aspectos más importantes ha sido entender la importancia de estructurar bien una aplicación desde el principio. La separación entre Laravel y React, el uso de rutas API, la validación de datos y la organización de componentes han sido claves para mantener el proyecto ordenado y comprensible.

También ha sido un proyecto útil para enfrentarse a problemas reales de desarrollo, como la gestión del estado en el frontend, la comunicación con el backend, la validación de formularios, el control de errores y la preparación del entorno de despliegue. Todo ello ha contribuido a mejorar la autonomía y la capacidad de resolución de problemas.

9.3. Posibles ampliaciones y mejoras

Como futuras mejoras, se podrían añadir funcionalidades como una pasarela de pago online, confirmaciones automáticas por correo electrónico, estadísticas de pedidos y reservas, gestión avanzada de mesas y horarios y un panel de administración más completo.

También sería interesante mejorar la parte de pruebas automatizadas para cubrir de forma más precisa los flujos principales de la aplicación, como el registro, las reservas y los pedidos. Esto permitiría detectar errores con mayor rapidez en futuras versiones.

En definitiva, Distrito Gourmet cumple con los objetivos principales del proyecto y deja una base preparada para seguir creciendo con nuevas funcionalidades.

10. Bibliografía

10.1. Artículos y apuntes

Apuntes oficiales del IES Serra Perenxisa: Material didáctico de los módulos de Desarrollo Web en Entorno Servidor (DWES) y Desarrollo Web en Entorno Cliente (DWEC). Han servido como base teórica fundamental para estructurar la arquitectura del proyecto y aplicar las buenas prácticas de programación.

Artículos técnicos: Consulta de diversos tutoriales y casos de estudio elaborados por la comunidad de desarrolladores sobre la integración específica de una API REST en Laravel con un frontend desacoplado en React.

Foros: Utilizados como apoyo puntual para la resolución de errores específicos durante el desarrollo, especialmente en la configuración del proxy inverso con Nginx y la contenerización con Docker.

10.2. Direcciones web

- **Laravel.** Documentación oficial de Laravel 12. Consultada para la estructura del backend, rutas API, controladores, validaciones, migraciones y uso de Eloquent ORM. Disponible en: <https://laravel.com/docs/12.x>
- **Laravel Sanctum.** Documentación oficial. Consultada para la autenticación mediante tokens Bearer en la API REST. Disponible en: <https://laravel.com/docs/12.x/sanctum>
- **React.** Documentación oficial. Consultada para la creación de componentes, hooks y estructura de la SPA del frontend. Disponible en: <https://react.dev/>
- **Vite.** Documentación oficial. Consultada para la configuración del entorno de desarrollo, compilación y empaquetado del frontend. Disponible en: <https://vite.dev/>
- **React Router.** Documentación oficial. Consultada para la gestión de rutas y navegación entre vistas de la aplicación. Disponible en: <https://reactrouter.com/>
- **Zustand.** Documentación oficial. Consultada para la gestión del estado global de usuario, sesión y carrito. Disponible en: <https://zustand.docs.pmnd.rs/>
- **Axios.** Documentación oficial. Consultada para la comunicación HTTP entre el frontend React y la API REST de Laravel. Disponible en: <https://axios-http.com/docs/intro>

- **Tailwind CSS.** Documentación oficial. Consultada para la maquetación responsive y la definición de estilos de la interfaz. Disponible en: <https://tailwindcss.com/docs>
- **GSAP.** Documentación oficial. Consultada para la implementación de animaciones e interacciones visuales en el frontend. Disponible en: <https://gsap.com/docs/v3/>
- **MySQL 8.0.** Documentación oficial. Consultada para el diseño físico de la base de datos, tipos de datos, claves primarias, claves foráneas e índices. Disponible en: <https://dev.mysql.com/doc/refman/8.0/en/>
- **Docker.** Documentación oficial. Consultada para la contenerización del backend, frontend y base de datos mediante Docker Compose. Disponible en: <https://docs.docker.com/>
- **Nginx.** Documentación oficial. Consultada para la configuración del servidor web del frontend y el enrutamiento de la aplicación SPA. Disponible en: <https://nginx.org/en/docs/>
- **MDN Web Docs.** Referencia técnica utilizada para consultar compatibilidad de HTML, CSS y JavaScript en navegadores modernos. Disponible en: <https://developer.mozilla.org/>
- **OWASP Foundation.** OWASP Top 10. Consultado como referencia para buenas prácticas de seguridad web, validación de entradas y prevención de vulnerabilidades comunes. Disponible en: <https://owasp.org/www-project-top-ten/>

11. Recursos del proyecto

Repositorio

El código fuente completo del proyecto está disponible en el siguiente repositorio:

<https://github.com/AVL05/distrito-gourmet>

Video de demostración

La demostración funcional de la aplicación puede verse en: <https://youtu.be/LWB3tT9npX4>